Module 4
Functions and Introduction to Pointers

Function

- A function is a group or block of statements that perform a particular task.
- The function can be classified into two categories:

1. Built in function or Library defined functions
2. User defined functions

Library defined functions:

- C Library that comes with C compiler has a collection of various functions which perform standard and predefined tasks.
- C language provides many built in functions such as,
  - Mathematical functions such as pow(x,y) , sqrt(x,y) etc.,
  - String manipulation functions such as strlen(),strcpy() etc.,
  - Memory management functions malloc(),calloc() etc.,

User defined function:

- C allows programmers to create their own functions called as user defined functions.
- These user defined functions are required because built in functions provided by C are not sufficient for performing customized functions.

Advantages of function:

- It reduces complexity of writing single large program by having a separate function for each subtask.
- Such smaller subprograms or functions are easier to test, debug and maintain.
- Set of instructions specifying the task performed by the function is written just once but can be used any number of times.

Elements of user defined functions/structure of a function:

The three elements of user defined functions are:-

1. Function declaration
2. Function definition
3. Function Call

Function Declaration / Function prototype:

- Like variables, all functions in a C program must be declared, before they are invoked. A function declaration (also known as function prototype) consists of four parts
- ***Function type or Return Type***
- ***Name of the Function***
- ***Parameter list.***
- ***Terminating semicolon.***

- Syntax of Function Declaration:

return_type function_name(parameter list with datatype);

Example: int add(int a, int b);

int – return type

add – function name

int a, int b – parameter list

Function Definition:

- A function definition, also known as function implementation shall include the following elements:

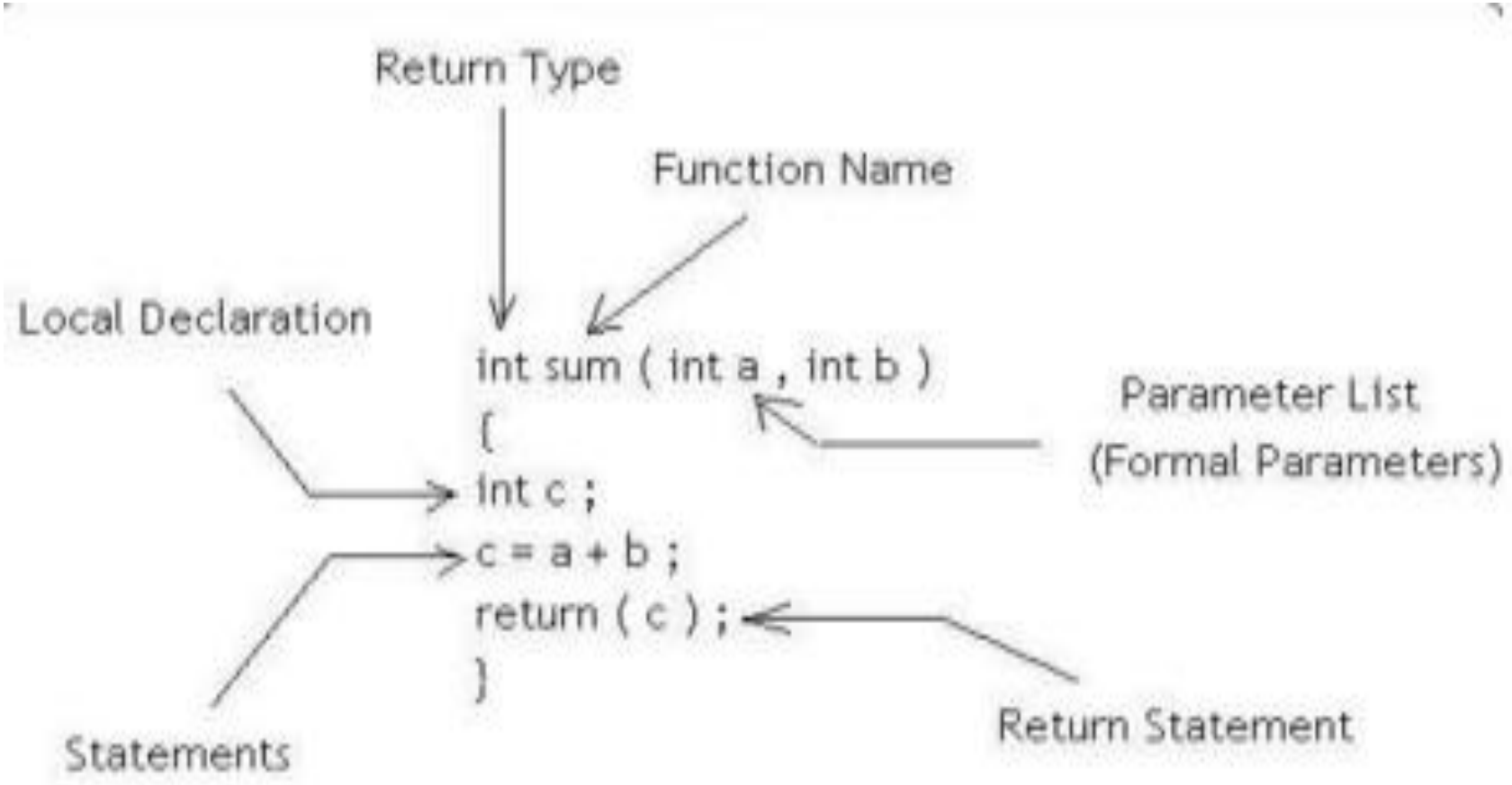
1. function name;
2. function type;
3. list of parameters;
4. local variable declarations;
5. function statements; and
6. a return statement.

- All the six elements are grouped into two parts, namely,
  - function header (First three elements); and
  - function body (Second three elements).

It provides the actual body of the function.

- It contains Executable Code (Executable statements).
- Syntax of function definition:

```
return_type function_name(parameter list with datatype)
{
        Local variable declarations;
        Statement(s);
        return value;
}
```


Return Type
Function Name
Local Declaration
int sum (int a , int b)
{
int c ;
c = a + b ;
return (c) ;
}
Parameter List
(Formal Parameters)
Statements
Return Statement

Function Call :

- A function can be called by simply using the function name followed by a list of actual parameters (or arguments) if any enclosed in parentheses.
- Syntax: function_name(parameters);

- Example: add(a,b);

Write a function to find sum of two numbers.

```
#include<stdio.h>
int add(int a, int b );          //function declaration

void main()
{
    int a,b,c;
    printf("\n Enter the values of a and b\n");
    scanf("%d %d", &a,&b);
    c=add(a, b );                //function call
    printf("sum is %d",c);
}

int add(int a, int b)            // function definition
{
    int sum;
    sum = a+b;
    return sum;
}
```

- Output

Enter the values of a and b

2

3

Sum is 5

- The function which calls another function is called as **calling function**

  Example: main( )

- The function which is called by another function is called as **called function**.

  Example: add( )

Formal and Actual parameters:

Formal Parameter:

- Parameter written in function definition and function declaration is called “formal parameter”.
- The formal parameters will receive the data sent by the calling program.
- Since these parameters formally represent the actual values and are not the actual values, they are called formal parameters

example

```
#include<stdio.h>
void display(int a);                    //function declaration
void main()
{
    int x=10;
    display (x);                        //function call
}
void display(int a)                     //function definition
{
    printf("%d",a);
}
```

Here a is the formal parameter

Actual Parameter:

- Parameter written in function call is called “actual parameter” because they are the actual values used for computation.

example

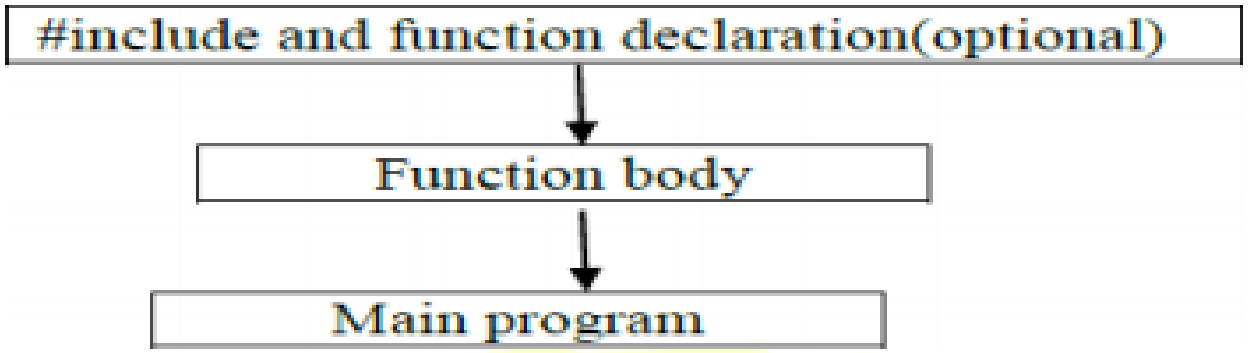
```
#include<stdio.h>
void display(int a);                    //function declaration
void main()
{
    int x=10;
    display (x);                        //function call
}
void display(int a)                     //function definition
{
    printf("%d",a);
}
```

Here x is the actual parameter

Location of Functions

- Location of function refers to the placement of the function definition with reference to the main function. There are mainly three ways to place a function in program.

- **Type1:** Placing a function definition first, followed by the entire main program. In this case defining or declaring function prototype can be omitted.

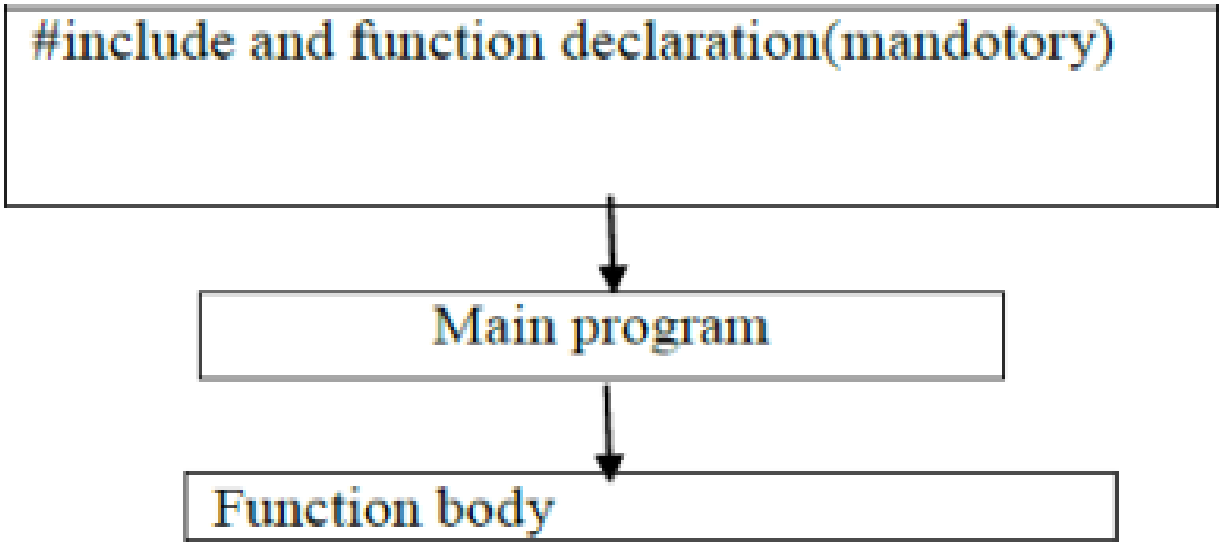


```
/* C program to add two numbers */
#include<stdio.h>
int add(int a, int b); //function declaration is optional

int add(int a, int b)
{
    int sum;
    sum =a+b;
    return sum;
}

void main( )
{
    int c, a=10,b=20;
    c= add(a, b);
    printf (sum is %d", c);
}
```

- **Type 2:** In this type the body of function (function definition) are placed after the main( ) program. Here one has to place the function prototype statement well before main( ) program appearance in order to assists C compiler for type checking and function call location with respect to function definition



```
/* C program to add two numbers */
#include<stdio.h>
int add(int a, int b);                    //function declaration is mandatory

void main( )
{
    int c, a=10,b=20;
    c= add(a, b);
    printf (sum is %d", c);
}

int add(int a, int b)
{
    int sum;
    sum =a+b;
    return sum;
}
```

- Type 3: Here the function body is placed in one file and the main() program with function prototypes in another file.
- Example: In the file say **add.h** the following function will be defined

```
int addition(int x, int y)
{
        int s;
        s=x+y;
        return s;
}
```

In the file say addition.c the following code has to be included

```
#include <stdio.h>
#include<add.h>              /* the header file which
                                includes the function definition */

void main()
{
int n1,n2,sum ;
printf("Enter the values of n1 and n2\n"); \
scanf("%d %d",&n1,&n2);
sum = addition(n1,n2);                  /* Function Call */
printf("Sum =%d \n ",sum);
}
```

Categories of user defined functions

1. Function with parameters and with return values.
2. Functions with parameters and without return values
3. Function without parameter and with return values.
4. Functions without parameters and without return values. (Void and parameter less Functions).

Function with parameters and with return values.

- Parameters are passed from calling function to the called function. Based on the received parameter values called function performs required action and returns a value back to the calling function.

```
//C program to find the cube of a given number
#include<stdio.h>
int cube(int k);

void main()
{
int n,
printf(“Enter a Number\n”);
scanf(“%d”,&n);
a=cube(n);
printf(“cube of %d is %d\n”,n,a);
}

int cube(int k)
{
int c;
c=k*k*k;
return c;
}
```

- Output

Enter a number

3

Cube of 3 is 27

Function with parameters and without return values

- Parameters are passed from calling function to called function and called function does not return a value. It just performs the specified action.

Example:
//C program to find the cube of a given number

```
#include<stdio.h>
void cube(int k);

void main()
{
    int n;
    printf("Enter a Number\n");
    scanf("%d",&n);
    cube(n);
}

void cube(int k)
{
    int c;
    c=k*k*k;
    printf("cube of %d is %d\n",k,c);
}
```

Function without parameters and with return values

- function does not receive any parameter from calling function. It takes the parameter within itself and performs specified action and returns a result to calling function.

```
#include<stdio.h>
int cube( );

void main()
{
    int a;
    a=cube( );
    printf(“cube is %d\n”,a);
}

int cube( )
{
    int n,c;
    printf(“Enter a Number\n”);
    scanf(“%d”,&n);
    c=n*n*n;
    return c;
}
```

Function without parameters and without return values

- No parameter is passed from the calling function to the called function. Called function does not return any values to the calling function. This type of function is also called as void and parameter less Functions).

```
#include<stdio.h>
void cube( );

void main()
{
     cube( );
}

void cube( )
{
    int n,c;
    printf(“Enter a Number\n”);
    scanf(“%d”,&n);
    c=n*n*n;
    printf(“cube of %d is %d\n”,n,c);
}
```

- Write a user defined function to find sum of 1st n natural numbers. Use it in the main() function to display the result

Argument Passing – call by value, call by reference

There are two ways of passing parameters to the function

Pass by Value (Call by value)

Pass by Reference (Call by Reference)

- **Pass by Value (Call by value):** Here the values of actual parameters are copied into formal parameters i.e. ***formal parameters contain only the copy of actual parameters***. So even if the value of the formal parameters changes in the called function the values of the actual parameters are not changed

```
#include<stdio.h>
void swap (int m , int n)                    // Formal Parameters m,n

{
int temp;
temp = m;
m = n;
n = temp;
}

void main()
{
int a,b;
 a=10, b=20;
printf("\n The values of a and b before calling swap\n");
printf("\n a=%d b= %d \n ",a,b);
swap(a,b);                                   //Actual Parameters a,b
printf("\n The values of a and b after calling swap()\n"); printf("\n a =%d and b =
%d \n",a,b);
}
```

```
The values of a and b before calling swap

a=10 b= 20

The values of a and b after calling swap()

a =10 and b = 20
```

- **Pass by Reference (Call by Reference) :**In pass by reference / address when a function is called the address of actual parameters are sent. In the called function the *formal parameters* should be declared as pointers with the same type as the *actual parameters*. Using these addresses the values of the actual parameters can be changed. This way of changing the actual parameters is called pass by addresses.

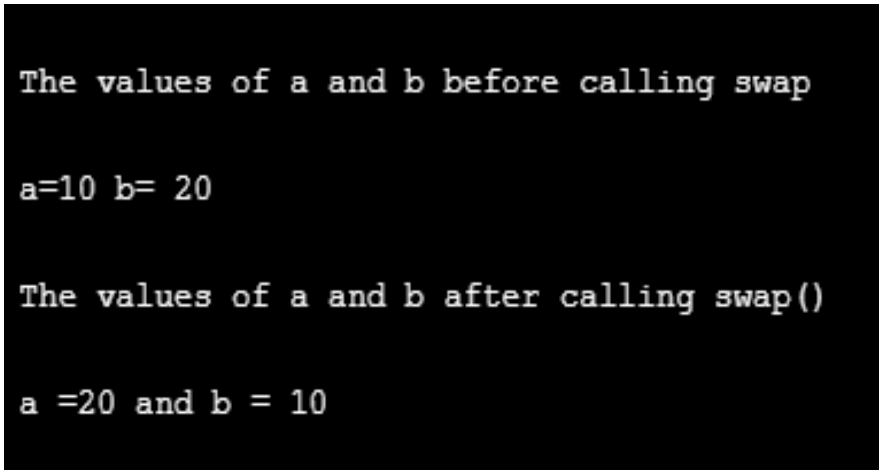
```
#include<stdio.h>
void swap (int *m , int *n)
{
    int temp;

    temp = *m;
   *m = *n;
   *n = temp;
}
void main()
{
int a,b;
a=10, b=20;
printf("\n The values of a and b before calling swap is  a=%d b= %d \n",a,b );

swap(&a,&b);

 printf("\n The values of a and b after calling swap()\n");

printf("\n a=%d and b = %d \n",a,b);
}
```



Arrays and Functions

- **Passing One Dimensional Arrays** : Like the values of simple variables it is also possible to pass the values of an array to a function. To pass a one dimensional array to a function it is sufficient to *list the name of the array* , without any subscripts , and *the size of the array* as arguments.

Example:- **int big(int a[ ], int n);**

Consider the problem of **finding the largest value in an array of elements**

```
#include<stdio.h>
int big(int a[ ],int n)
{
        int i,max;
        max = a[0] ;
        for(i =1;i<n ;i++)
        {
                if(a[i]>max)
                max = a[i];
        }
        return max;
}
```

```
void main()
{
int i,n,a[100], large;
printf(“ Enter the value of n \n”);
scanf(“%d”, &n);
printf(“ Enter %d elements \n”, n);
for(i=0;i<n;i++)
{
        scanf(“%d”,&a[i]);
}
large= big(a,n);
printf(“\n The largest element is %d\n”,large);
}
```

Write a C program to find sum of array elements using functions

```
#include<stdio.h>
int add(int a[],int n)
{
   int i,sum=0;
   for(i=0;i<n;i++)
  {
      sum=sum+a[i];
   }
return  sum;
}
```

```
void main()
{
 int a[100],n,i,sum;
 printf("Enter the number of elements:\n");
 scanf("%d",&n);
 printf("Enter %d elements :\n", n);
 for(i=0;i<n;i++)
 {
    scanf("%d", &a[i]);
 }
sum=add(a,n);
printf("sum of %d elements are %d\n",n, sum);
}
```

- **Three Rules to pass an array to a function:**

1. The function must be called by passing only the name of thearray.
2. In the function definition the formal parameter must be an array type, the size of the array does not need to be specified.
3. The function prototype must show that the argument in an array.

- **Automatic variables-**Automatic variables are declared inside a function in which they are to be utilized. They are created when the function is called and destroyed automatically when the function is exited, hence the name automatic. Automatic variables are therefore private (or local) to the function in which they are declared.

- **External variables** Variables that are both alive and active throughout the entire program are known as external variables. They are also known as global variables. Unlike local variables, global variables can be accessed by any function in the program

- **Static variables** As the name suggests, the value of static variables persists until the end of the program. A variable can be declared static using the keyword static A static variable may be either an internal type or an external type depending on the place of declaration

Storage Class	*Where declared*	*Visibility (Active)*	*Lifetime (Alive)*
None	Before all functions in a file (may be initialized)	Entire file plus other files where variable is declared with **extern**	Entire program (Global)
extern	Before all functions in a file (cannot be initialized) **extern** and the file where originally declared as global.	Entire file plus other files where variable is declared	Global
static	Before all functions in a file	Only in that file	Global
None or **auto**	Inside a function (or a block)	Only in that function or block	Until end of function or block
register	Inside a function or block	Only in that function or block	Until end of function or block
static	Inside a function	Only in that function	Global

RECURSION

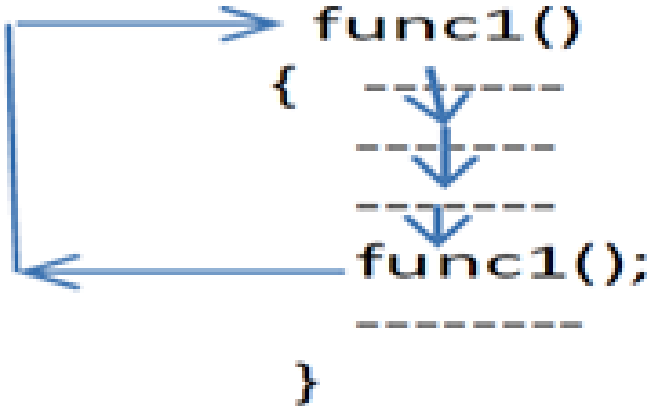
Introduction

- Recursion refers to the process where a function calls itself repeatedly until a certain exit condition is met.
- A function which calls itself repeatedly until a certain exit condition is met is called recursive function.

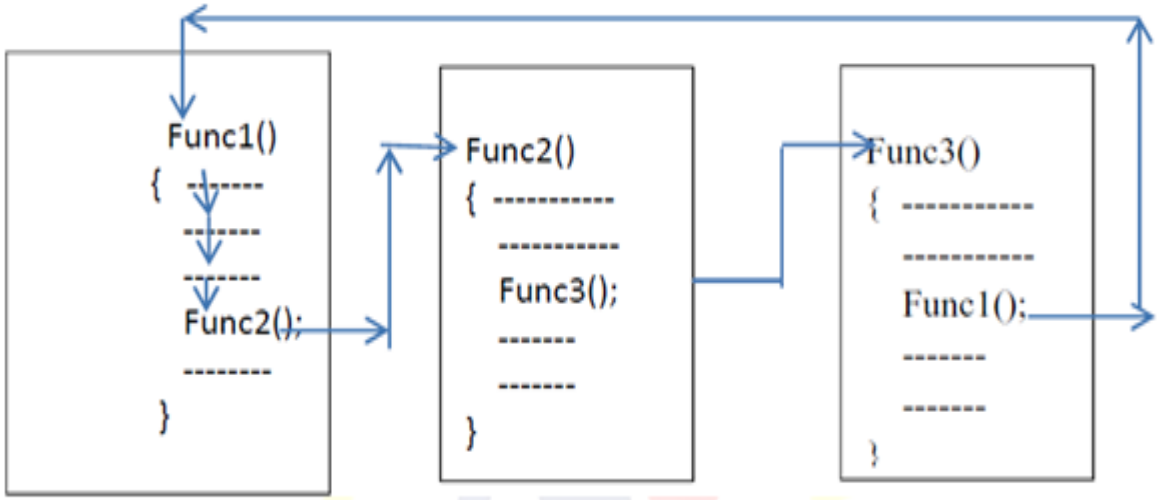
- There are two types of recursion as given below:
- DirectRecursion

- IndirectRecursion

- **Direct Recursion :**It refers to a process where a function calls itself directly as illustrated in the figure below :



- **Indirect Recursion**: It refers to a process where a function calls another function and that function calls back the original function. The process of indirect recursion takes place as illustrated in the figure below:



Here the Func1() calls Func2() and Func2() calls Func3() and Func3() calls Func1().

1. Write a C program to find the factorial of a number using recursion method

- Consider we need to find 4!
- We know that 4!= 4*3*2*1=24

4!=4***3*2*1**

=4*3! =>n*(n-1)! , if we expand

=4*3*2!

= 4*3*2*1!

=4*3*2*1*0!

=4*3*2*1*1 (we know 0!=1 , so that is the terminating condition)

= 24

- Therefore for a factorial program the design is:

Recurrence relation would be,

$$fact(n) = \begin{cases} 1 & \text{if } n==0 \\ n*fact(n-1) & \text{if } n>0 \end{cases}$$

- n=5 ; fact(n)=n*fact(n-1)
- fact(5) = 5 * fact(5-1)

= 5* fact(4) → (function calling itself, again **n*fact(n-1)**)

= 5* 4*fact(3)

= 5*4*3*fact(2)

= 5*4*3*2*fact(1)

= 5*4*3*2*1*fact(0) (stopping condition 0!=1)

= 5*4*3*2*1*1

=120

```
#include<stdio.h>
int fact(int n);

void main( )
{
    int k,a;
    printf(“enter the value of number \n” );
    scanf(“%d”,&k);
    a= fact(k);
    printf(“the factorial of a %d is %d”,k,a);
}
```

```
int fact(int n)
{
   if(n==0)
        return 1;
 else
        return( n * fact(n-1));
}
```

2. Write a C program to generate n Fibonacci series using recursion

- Recurrence relation would be,

$$fib(n) = \begin{cases} 0 & \text{if } n=1 \\ 1 & \text{if } n=2 \\ fib(n-1)+fib(n-2) & \text{if } n>0 \end{cases}$$

```
#include<stdio.h>
int fib(int n);

void main( )
{
    int k,i,x;
    printf("enter the value of n" );
     scanf("%d",&k);
    printf("Fibonacci series is");
     for(i=1;i<=k;i++)
    {
            x= fib(i);                    //function call
            printf(" %d\t",x);
    }
}
```

```
int fib(int n)
{
     if(n==1)
          return  0;
    else if(n==2)
          return  1;
    else
          return(fib(n-1)+fib(n-2));  //recursive function call
}
```

- Output

enter the value of n

4

Fibonacci series is

0 1 1 2

Let n= 4

for (i=1;i<n;i++) printf("%d\t", fib(i));

		fib(1)	fib(2)	fib(3)	fib(4)
When i=1	fib(1)	0			
When i=2	fib(2)	0	1		
When i=3	fib(3)	0	1	fib(2)+fib(1)	
		0	1	1+0	
		0	1	1	
When i=4	fib(4)	0	1	1	fib(3)+fib(2)
		0	1	1	fib(2)+fib(1)+1
		0	1	1	1+0+1
		0	**1**	**1**	**2**

3. Write a C program to compute polynomial co-efficient nCr using recursion

- if n>r then

$$^{n}c_{r} = n! / (r! * (n-r)!)$$

- The above equation can be given as :-

$$^{n}c_{r} = fact(n) / (fact(r) * fact(n-r))$$

```
#include<stdio.h>
int fact(int n);

void main( )
{
    int n,r,res;
    printf(“enter the value of n and r \n” );
    scanf(“%d %d”,&n,&r);
    if (n>r)
    {
        res  =  fact(n)  /  (fact(r) * fact(n-r));
        printf (“the result is  %d”, res);
    }
     else
    {
        printf (“n should be greater than r”);
    }
}
```

```
int fact(int n)
{
   if(n==0)
        return 1;
 else
        return( n * fact(n-1));
}
```

- Output

enter the value of n and r

4

2

Result is 6

analysis

```
if (n>r)
{
res  =  fact(n)  /  (fact(r) * fact(n-r));
}
```

```
int fact(int n)     //function definition
{
       if(n==0)
                    return 1;
 else
                    return( n * fact(n-1));
}
```

Let n=4 r=2

If(n>r) => (4>2) true

res = fact(n) / (fact(r) * fact(n-r));

= fact (4) / (fact(2) * fact(2));

- Numerator fact(4) function will be called and control moves to function definition with n = 4 and returns 24
- In Denominator fact(2) function will be called control moves to function definition with n =2 and returns 2
- again fact(2) function will be called control moves to function definition with n =2 and returns 2

- So **res = 24/(2*2)**

  = 24/4

  = 6

Write a program using function to check if number is prime or not. Functions should return 0 if the number is prime ,1 if not prime number

```
#include <stdio.h>
void main()
{
        int num,res=0;
        printf("\nENTER A NUMBER: ");
        scanf("%d",&num);
        res=prime(num);
        if(res==0)
                printf("\n%d IS A PRIME NUMBER",num);
        else
                printf("\n%d IS NOT A PRIME NUMBER",num);
}
```

```
int prime(int n)
{
        int i;
        for(i=2;i<=n/2;i++)
        {
                if(n%i!=0)
                        continue;
                else
                        return 1;
        }
        return 0;
}
```

POINTERS

What is Pointer??

- A pointer is a variable whose value is the **address of another variable.**
- Pointers contain memory addresses as their values.



Pointer Declaration:

- Pointer variables should be declared before they are used.

Syntax: datatype *pointer_name;

```
Eg:     int *p;        /*pointer to an integer */
        float *f;      /*pointer to float*/
```

int i=3;

Address of ‘i’

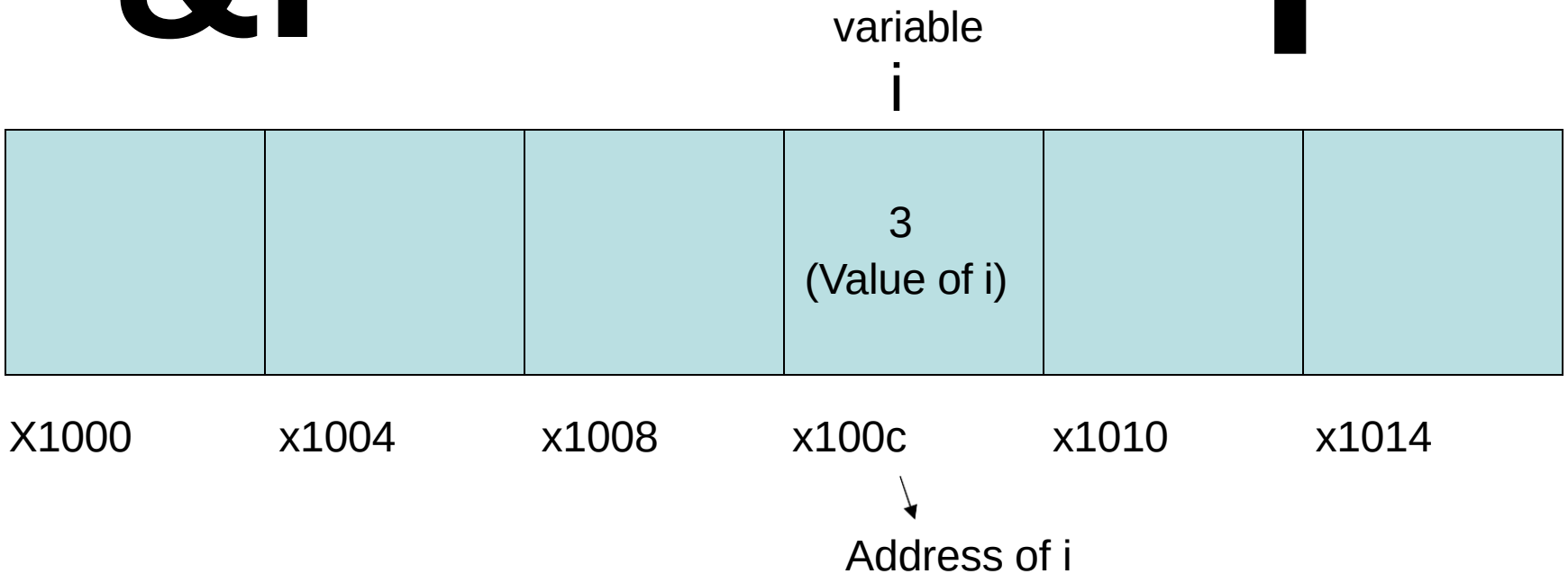
‘&i’

Value of ‘i’

‘*i’

*** is called dereference operator**

& is called reference or address operator.



The value ‘3’ is saved in the memory location ‘x100c’

Initialization of Pointer :

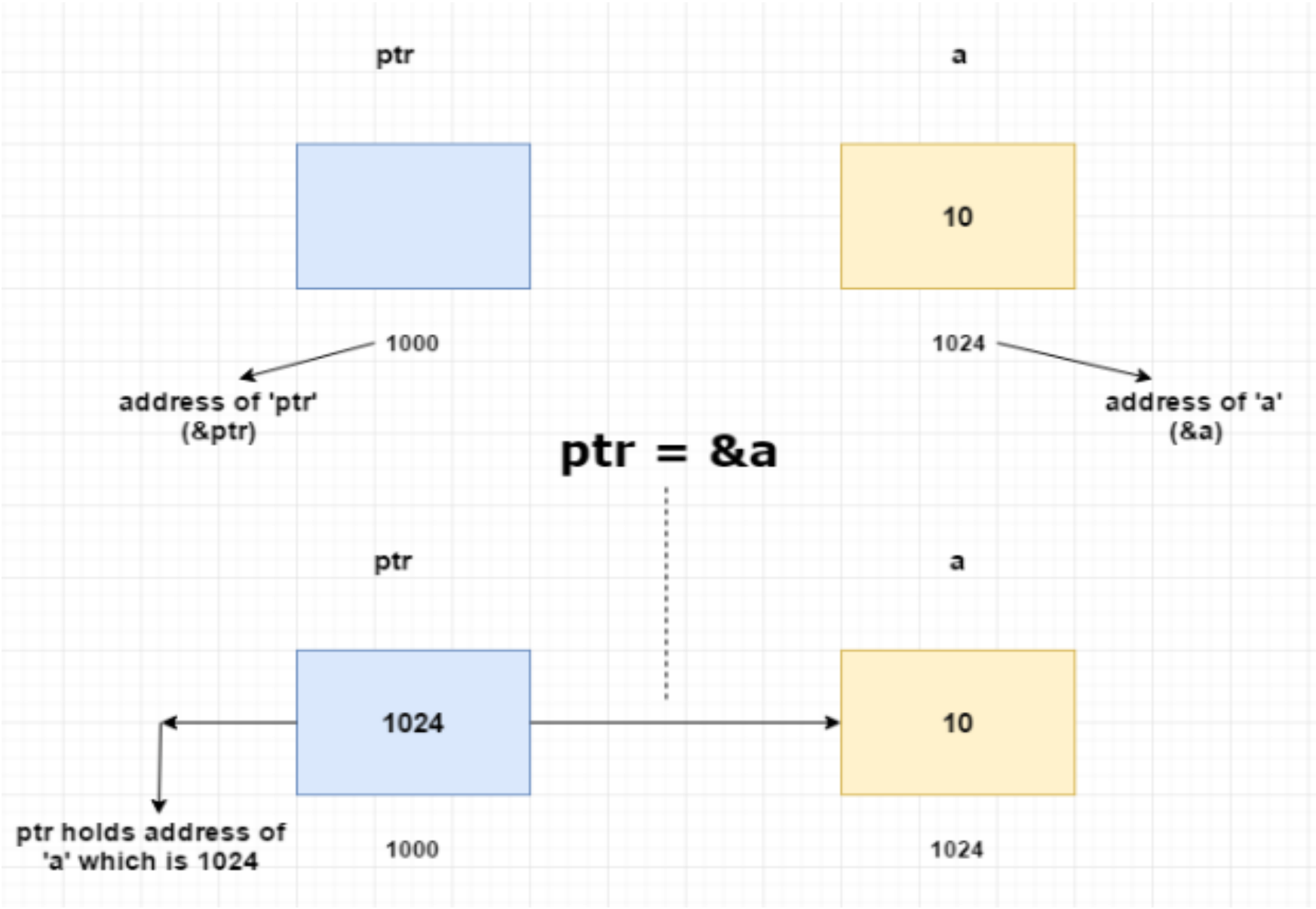
- **It** is the process of assigning an address to the pointer variable.

- **Syntax: pointer_name =&variable_name;**

Eg: int a=10;

int *p ; //pointer declaration

p= &a; //Initialization of pointer variable


ptr
a
10
1000
1024
address of 'ptr'
(&ptr)
address of 'a'
(&a)
ptr = &a
ptr
a
1024
10
ptr holds address of
'a' which is 1024
1000
1024

EXAMPLE

Remember! The content of the memory location referenced by a pointer is obtained using the "*" operator (this is called ***dereferencing the pointer). Thus, *px refers to the value of x which is 6.5.***

px is a pointer to float type; meaning : it would contain an address of float type object

x = 6.5;

px= &x;

Store the address of x into px

If the address of x is 0xFFFFF then what would be assigned to px?

Example program for pointer

```
#include<stdio.h>
void main()
{
int a=10;

int  *ptr;

ptr=&a;

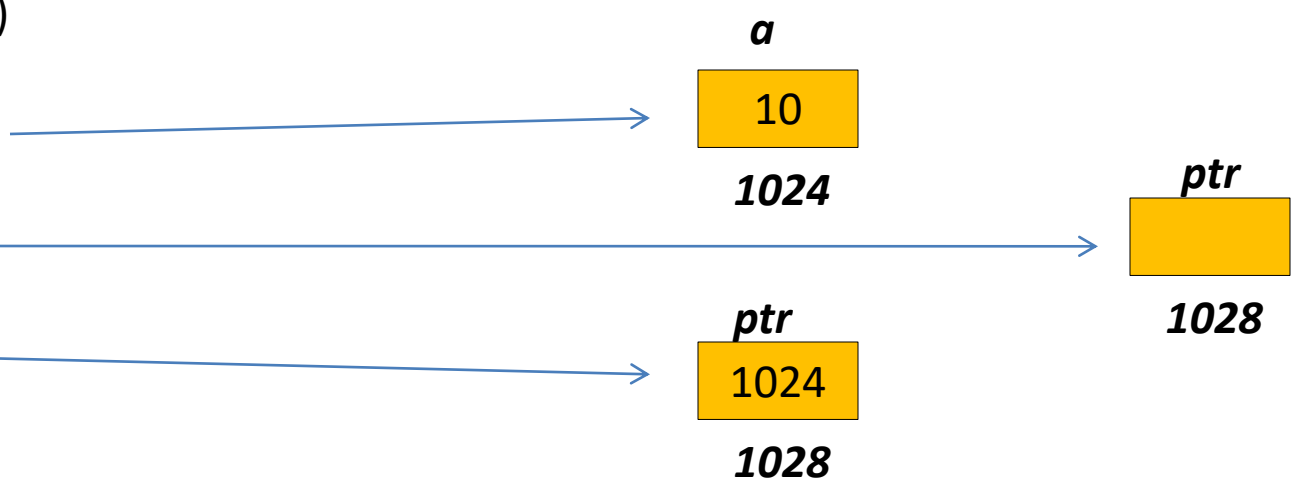
printf("%d\n",a);

printf("%d\n",&a);

printf("%d\n“, *ptr);

printf("%d\n",ptr);

}
```



Prints value of a => 10

Prints address of a => 1024

Prints value of variable that ptr is pointing to (value at 1024) => 10

Prints value stored in ptr =>1024

Output
10
1024
10
1024

OUTPUT ???

```
#include<stdio.h>
void main()
{
int a=10, b=15;

int  *p1, *p2;

p1=&a;

p2=&b;

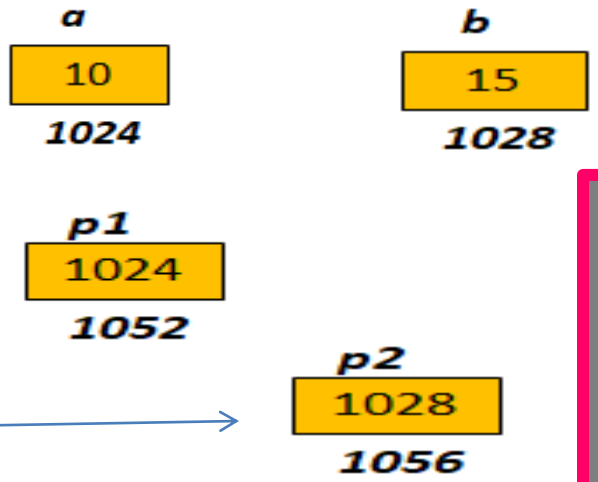
printf("%d\n",a);

printf("%d\n",p1);

printf("%d\n", *p1);

printf("%d\n",p2);

Printf ("%d\n", *p2)
}
```



Output
10
1024
10
1028
15

Prints value of a => 10

p1 contains address of a =>1024

Prints value of variable pointed by p1 => 10

p2 contains the address of b => 1028

Prints value of variable pointed by p2 (it points to b) => 15

Program where one pointer point to more than one variable

```
#include<stdio.h>
void main()
{
    int a=30;
    int b=40;
    int c=50;
    int *p;

    p=&a;
    printf("The value of a is %d\n",*p);

    p=&b;
    printf("The value of b is %d\n",*p);

    p=&c;
    printf("The value of c is %d\n",*p);
}
```

Output:

The value of a is 30

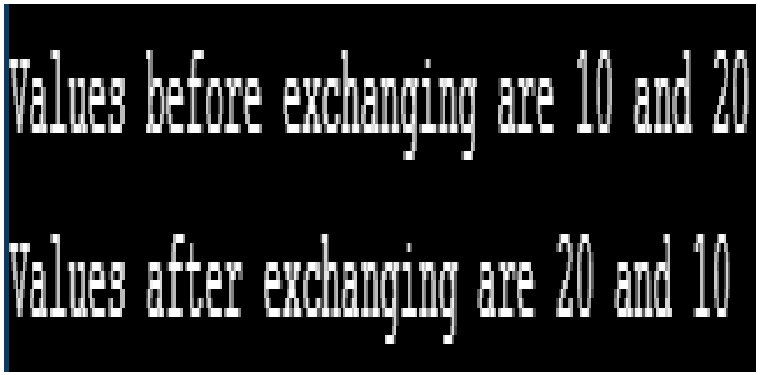
The value of b is 40

The value of c is 50

Pointers and Functions

Write a program to exchange/swap two numbers using pointers

```
#include <stdio.h>
void exchange(int *p, int *q)
{
   int temp;
   temp=*p;
   *p=*q;
   *q=temp;
}
void main()
{
  int a=10,b=20;
  printf("Values before exchanging are %d and %d\n",a,b);
  exchange(&a,&b);
  printf("Values after exchanging are %d and %d\n",a,b);
}
```



Pointer expression

- Like other variables, pointer variables can be used in expressions.
- if p1 and p2 are properly declared and initialized pointers, then the following statements are valid:
- y = *p1 * *p2; same as (*p1) * (*p2)
- sum = sum + *p1;

OUTPUT??

```
main()
 {
 int a, b, *p1, *p2, x, y, z;
 a = 12;
 b = 4;
 p1 = &a;
 p2 = &b;
 x = *p1 *  *p2 – 6;
printf(“a = %d, b = %d\n”, a, b);
 printf(“x = %d\n”, x);
 *p2 = *p2 + 3;
 *p1 = *p2 – 5;
 z = *p1 * *p2 – 6;
printf(“a = %d, b = %d\n”, a, b);
 printf(“ z = %d\n”, z);
 }
```

OUTPUT

a = 12, b = 4

x = 42

a = 2, b = 7, z = 8

Pointer Arithmetic

Increment operator on Pointer

- when we increment a pointer, its value is increased by the 'length' of the data type that it points to. This length called the scale factor

Final value of pointer = current value of the pointer + (integer value * size of the data type)

Eg:

int *ptr;

If ptr=1000,

ptr++ = ptr + (1*size of int)

= 1000+(1*2)=1002

float *ptr;

If ptr=1000,

ptr++ = ptr + (1*size of float)

= 1000+(1*4)=1004

```
#include<stdio.h>
main()
{
    int a,*p;
    p=&a;
    printf("value of pointer p before decrementation %u ", p);
    p--;
    printf("value of pointer p after decrementation %u ", p);
}
```

Output

value of pointer p before decrementation: 3472147

value of pointer p after decrementation : 3472145

Pointer Addition:

- Final value of a pointer = current value of the pointer + integer number * sizeof(data_type);

```
#include<stdio.h>
main()
{
  int a,*p;
  p=&a;
  printf("value of pointer p before addition %u ", p);
  p=p+5;
  printf("value of pointer p after addition %u ", p);
}
```

Output:

value of pointer p before addition: 3472147

value of pointer p after addition : 3472157

- Note: final value = 3472147 + (5 * sizeof int)
- =3472147 + (5 * 2)
- =3472157

Rules of Pointer Operations

The following rules apply when performing operations on pointer variables:

1. A pointer variable can be assigned the address of another variable.
2. A pointer variable can be assigned the values of another pointer variable.
3. A pointer variable can be initialized with NULL or zero value.
4. A pointer variable can be pre-fixed or post-fixed with increment or decrement operators.
5. An integer value may be added or subtracted from a pointer variable.
6. When two pointers point to the same array, one pointer variable can be subtracted from another.
7. When two pointers point to the objects of the same data types, they can be compared using relational operators.
8. A pointer variable cannot be multiplied by a constant.
9. Two pointer variables cannot be added.
10. A value cannot be assigned to an arbitrary address (i.e., &x = 10; is illegal).

Pointer to Pointer (chain of pointers)

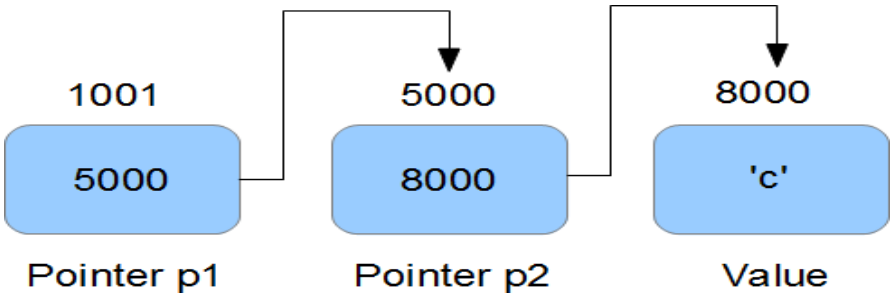
Pointer that points to another pointer is called as pointer to pointer.

Declaration:

Syntax: datatype **pointer_name;

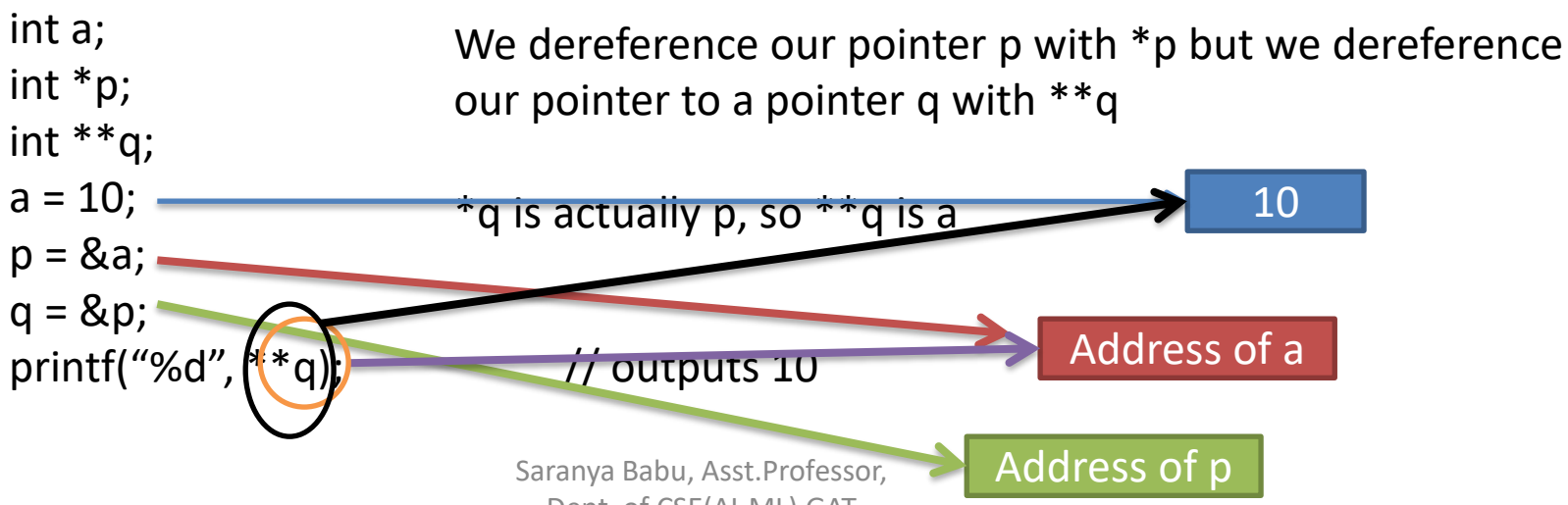
Initialization:

pointer_name = & another_pointer_name



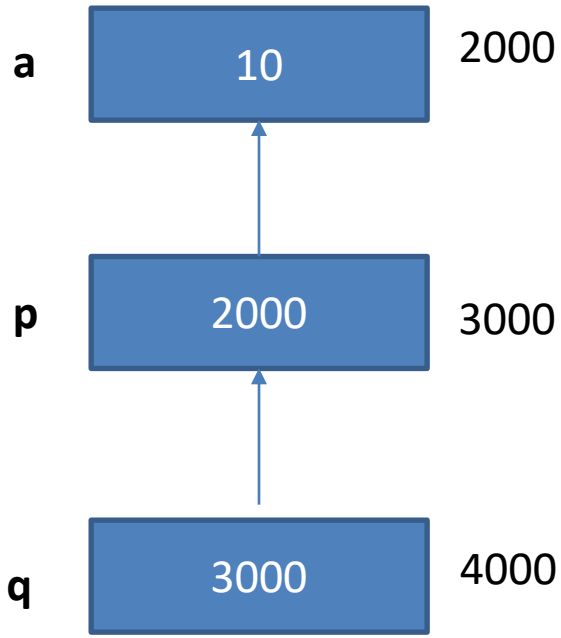
Pointers to Pointers

- As indicated in the last slide, we can have an array of arrays which is really an array of pointers or pointers to pointers
  - We may wish to use pointers to pointers outside of arrays as well, although it is more common that pointers to pointers represent array of pointers
  - Consider the following:



Example Program

```
#include<stdio.h>
void main()
{
    int a=10;
    int *p;
    int **q;
    p=&a;
    q=&p;
    printf("%d\n",a);
    printf("%d\n",*p);
    printf("%d\n",**q);
}
```



Output:
10
10
10

Difference between a pointer variable and a normal variable :

Sl. NO	Pointer Variable	Normal Variable
1	A pointer variable holds address	A normal variable holds data
2	The general format of declaring a pointer : ***data_type *pointer_name;***	The general format of declaring a variable is : ***data_type variable_name;***
3	The general format for initialization of pointer variable: ***pointer_name =&variable***	The general format for initialization of variable : ***variable_name = value;***
4	variable is essential to access data.	Data can be accessed without dereferencing a normal variable .

Pointers and Arrays

Explain how pointers and arrays are related with example.

Pointers can be used to

- Accessing array element
- Reading an array
- Printing an array

Write a program to find sum , mean and std . deviation of values stored in an array.

- When an array is declared the name of array always points to address of the first element and this address is called base address.
- Syntax for using pointers in array:

```
data_type *ptr;
ptr=array_name;
```

Eg:

```
int *p;
 int a[10];
  p=a;
```

- Note: & is not required

Example

```
#include<stdio.h>
main()
{
   char str[20] = “PCD”;
    char *ptr;
     ptr= str;
   printf(“%s \n”,str);
}
```

Advantages of pointers

☺ Pointers provide direct access to memory.

☺ Needs less memory space

☺ Reduces complexity of the program.

☺ Provides an alternate way to access array elements .

☺ Pointers can be used to pass information back and forth between the calling function and called function.

Disadvantages of pointers

☹ Pointers are not safe and insecure because values of other variables can be accessed without permission.

☹ Uninitialized pointers might cause program to crash.

☹ Pointers are slower than normal variables.

☹ If pointers are updated with incorrect values, it might lead to memory corruption.